

In Next.js, "fallback" rewrites are executed when a request URL does not match any static file, dynamic route, or `afterFiles` rewrite. This is useful for proxying to an external service as a last resort, after all internal Next.js routes have been checked.  Next.js +1

Fallback proxy rewrite rules are defined in your `next.config.js` (or `next.config.mjs`) file within the `async rewrites()` function. 

How to Implement Fallback Proxy Rewrites

1. **Open or create** the `next.config.js` file in the root of your project.
2. **Add the** `rewrites` **function** and return an object containing the `fallback` array.
3. **Define your fallback rule(s)** in the `fallback` array. 

Example `next.config.js`

javascript



```
/** @type {import('next').NextConfig} */
const nextConfig = {
  async rewrites() {
    return {
      // These rewrites are checked after static files and dynamic routes
      // and before the 404 page is rendered.
      fallback: [
        {
          source: '/*:path*',
          destination: `https://api.yourexternaldomain.com*`, // Proxy to an external API
        },
      ],
    };
  },
};

module.exports = nextConfig;
```

[Source: Next.js Documentation on Rewrites](#)

Key Considerations

- **Execution Order:** Fallback rewrites are the last to be checked by Next.js before rendering a 404 page. This ensures that your application's pages take precedence.

- **Proxy Behavior:** Rewrites act as a URL proxy, meaning the user's browser URL remains the same while content from the destination is served internally.
- **External Destinations:** When proxying to an external domain, you must use an **absolute URL** in the `destination` field.
- **Parameters:** Path parameters (e.g., `:path*`) can be used to capture segments of the incoming URL and pass them to the destination.
- **Development Server:** After changing `next.config.js`, you must **restart your development server** for the changes to take effect.  Next.js +4